Campus Event Management Platform

CS 202C course project ▪ IIT Dharwad

Installation & Operations Guide

Installing, running and maintaining Campus Events

Author	Prajyot (MC25B012)
Course	CS 202C, Indian Institute of Technology Dharwad
Document	CEMP-IG-01
Version	1.0, November 2026
Software	Flask + SQLite web application (Python 3.10+)

Part of the project documentation set: Project Report • Presentation Slides • User Manual • Installation & Operations Guide • Developer Guide • Test Plan & Report. Source: `docs/` and `report/` in the project repository.

Contents

1 Overview **2**

2 System requirements **2**

3 Installation **2**
3.1 Get the code . . . 2
3.2 Run the setup script . . . 3
3.3 Admin account . . . 3
3.4 Load demo data (optional) . . . 3
3.5 Start the application . . . 3
3.6 Verify the installation . . . 4

4 Script reference **4**

5 Configuration **4**
5.1 Environment variables . . . 4
5.2 Built-in settings . . . 5

6 Flask command-line commands **5**

7 Running without Bash **5**

8 Database operations **5**
8.1 Where the data lives . . . 5
8.2 Back up and restore . . . 5
8.3 Reset . . . 6
8.4 Inspect the data . . . 6

9 Building the documentation **6**

10 Troubleshooting **6**

11 Deployment notes **7**

12 Uninstalling **7**

1 Overview

This guide is for whoever installs and runs the Campus Event Management Platform: a lab assistant, an examiner, or a developer setting up their own copy. It covers system requirements, installation, day-to-day commands, configuration, the database, and solutions to common problems.

The application is a Flask web app that stores its data in a single SQLite file. Everything is driven by Bash scripts in `scripts/`, which work in **Git Bash on Windows** and in any Bash shell on **Linux or macOS**.

Quick start

```
git clone https://github.com/PrajyotKira/Campus-Event-Management-Platform.git
cd Campus-Event-Management-Platform
./scripts/setup.sh      # venv, packages, database, indexes, admin account
./scripts/seed.sh       # optional: demo data
./scripts/run.sh        # open http://127.0.0.1:5000
```

2 System requirements

Component	Version	Notes
Python	3.10 or newer	From https://www.python.org/. On Windows, tick "Add Python to PATH". Developed with Python 3.14; also tested on Python 3.13 (Linux).
Bash	4 or newer	Windows: Git Bash, installed with Git for Windows (https://git-scm.com/). Linux/macOS: built in.
Git	any recent	To clone the repository.
Web browser	any modern	Chrome, Firefox, Edge or Safari.
LaTeX	optional	TeX Live or MiKTeX, only to rebuild the PDF documentation. Overleaf also works.

Table 1: Requirements.

SQLite needs no installation: it is built into Python's standard library. The Python packages are listed in `requirements.txt` and installed automatically:

Package	Version range	Used for
Flask	`>=3.0,<4.0`	web framework, Jinja2 templates, CLI commands
Flask-Login	`>=0.6.3`	login sessions
pytest	`>=8.0`	automated tests
Faker	`>=25.0`	realistic demo data
snakeviz	`>=2.2`	viewing profiler output in the browser

Table 2: Python dependencies (Werkzeug, Jinja2 and Click come with Flask).

3 Installation

3.1 Get the code

```
git clone https://github.com/PrajyotKira/Campus-Event-Management-Platform.git
cd Campus-Event-Management-Platform
```

You can also download the repository as a ZIP from GitHub and extract it. If the folder name contains spaces, keep the quotes when you `cd` into it.

3.2 Run the setup script

```
./scripts/setup.sh
```

It performs six steps and prints each one:

1. **Find Python.** Tries `python3`, then `python`, by actually running `-version` (on Windows `python3` can be a non-working Microsoft Store shortcut).
2. **Create the virtual environment** in `venv/` (skipped if it already exists).
3. **Install requirements** with `pip`.
4. **Create the database** `instance/campus.db` from `sql/schema.sql`.
5. **Add indexes** from `sql/indexes.sql`.
6. **Create the admin account** (default `admin@campus.local` / `admin123`).

Warning

Running `setup.sh` again **deletes all data**, because step 4 recreates every table. Back up `instance/campus.db` first if you need it (Section 8.2).

3.3 Admin account

Choose your own admin login with environment variables:

```
ADMIN_NAME="Event Office" ADMIN_EMAIL=events@iitdh.ac.in ADMIN_PASSWORD='S3cure!pw' ./scripts/
    setup.sh
```

To add another admin later without wiping the database:

```
source venv/Scripts/activate      # Windows Git Bash  (Linux/macOS: source venv/bin/activate)
python -m flask --app app create-admin --name "Second Admin" --email second@campus.local --
    password secret
```

If the email already exists the command does nothing and says so.

3.4 Load demo data (optional)

```
./scripts/seed.sh          # 50 events
./scripts/seed.sh 200      # 200 events
```

This replaces all venues, events, registrations and student accounts (admins are kept) with: 10 real-sounding campus venues, 100 students (`student1@campus.local` to `student100@campus.local`, password `student123`), the requested number of events spread over the next 60 days without any venue clash, and random registrations that never exceed an event's capacity. The random seed is fixed, so the same command always creates the same data.

3.5 Start the application

```
./scripts/run.sh              # http://127.0.0.1:5000
PORT=8000 ./scripts/run.sh    # another port
```

Open the printed address in a browser. Press `Ctrl+C` in the terminal to stop the server. The server runs in debug mode: it reloads automatically when code changes and shows detailed error pages, which is right for development and demonstrations.

3.6 Verify the installation

1. Run `./scripts/test.sh`. The last line should read `45 passed`.
2. Open the site, log in as the admin and open **Admin** → **Reports**.
3. Log in as `student1@campus.local` in a private window and register for an event.

4 Script reference

All scripts live in `scripts/`, start with `set -euo pipefail` (stop on the first error, treat unset variables as errors, fail a pipeline if any part fails) and can be run from any folder.

Command	What it does
`./scripts/setup.sh`	One-time setup (above). Variables: `ADMIN_NAME`, `ADMIN_EMAIL`, `ADMIN_PASSWORD`.
`./scripts/run.sh`	Activates the venv, checks the database exists, starts Flask in debug mode. Variable: `PORT` (default 5000).
`./scripts/seed.sh [N]`	Replaces non-admin data with demo data with N events (default 50). N must be a positive whole number.
`./scripts/test.sh [args]`	Runs `pytest -v`. Extra arguments go to pytest, e.g. `./scripts/test.sh -k clash` or `./scripts/test.sh tests/test_admin.py`.
`./scripts/profile.sh <label> [N]`	Builds a separate database `profiling/profile.db` with N events (default 2000), profiles the reports and search, saves results in `profiling/<label>/`. Variables: `RUNS` (timed runs per workload, default 3), `APPLY_INDEXES` (1 = add indexes, 0 = skip).
`./scripts/build_docs.sh`	Compiles every LaTeX document in `report/` and `docs/` to PDF with `latexmk` or `pdflatex`.
`scripts/common.sh`	Not run directly. Shared functions `find_python` and `activate_venv`, and the variables `PROJECT_ROOT` and `VENV_DIR`.

5 Configuration

5.1 Environment variables

Variable	Default	Meaning
`SECRET_KEY`	`dev-change-me`	Signs the login cookie. **Must** be changed for any real deployment.
`CAMPUS_DB`	`instance/campus.db`	Path of the SQLite database file. `profile.sh` sets it to `profiling/profile.db`.
`PORT`	`5000`	Port used by `run.sh`.
`ADMIN_NAME`	`Admin`	Admin display name for `setup.sh`.
`ADMIN_EMAIL`	`admin@campus.local`	Admin login for `setup.sh`.
`ADMIN_PASSWORD`	`admin123`	Admin password for `setup.sh`.
`RUNS`	`3`	Timed runs per workload in `profile.sh`.
`APPLY_INDEXES`	`1`	Whether `profile.sh` adds the indexes.

Table 3: Settings read from the environment.

Set a variable for a single command by writing it in front, e.g. `PORT=8000 ./scripts/run.sh`, or for the whole terminal session with `export SECRET_KEY=...`.

5.2 Built-in settings

`create_app()` in `app/__init__.py` also sets `SESSION_COOKIE_SAMESITE="Lax"`, so the browser does not send the login cookie with form submissions from other websites. The list of event categories (*Technical, Cultural, Sports, Workshop, Seminar*) is the `CATEGORIES` list in `app/events.py`; edit it there to add a category.

6 Flask command-line commands

The scripts call these commands; you can also run them yourself after activating the virtual environment. They are always run as `python -m flask -app app <command>` because the `flask.exe` launcher can be blocked on Windows.

Command	Effect
`init-db`	Drops and recreates all tables from `sql/schema.sql`. **Deletes all data.**
`add-indexes`	Creates the indexes in `sql/indexes.sql` (safe to repeat) and lists them.
`create-admin -name N -email E -password P`	Adds an admin account; skips an existing email.
`seed -events N -students M`	Replaces non-admin data with demo data (defaults 50 and 100). Needs an admin to exist.
`run -debug -port P`	Starts the development server (what `run.sh` does).
`routes`	Lists every URL the app answers to (built into Flask).

Table 4: Flask CLI commands.

7 Running without Bash

If Bash is not available (for example in Windows PowerShell or Command Prompt), run the same steps by hand from the project folder:

```
python -m venv venv
venv\Scripts\Activate.ps1          # Command Prompt: venv\Scripts\activate.bat
python -m pip install -r requirements.txt
python -m flask --app app init-db
python -m flask --app app add-indexes
python -m flask --app app create-admin
python -m flask --app app seed --events 50      # optional demo data
python -m flask --app app run --debug           # http://127.0.0.1:5000
python -m pytest -v                             # tests
```

Listing 1: Manual setup in PowerShell.

If PowerShell refuses to run `Activate.ps1`, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once.

8 Database operations

8.1 Where the data lives

All data is in one file, `instance/campus.db`. It is listed in `.gitignore`, so it is never committed. The `instance/` folder is created automatically.

8.2 Back up and restore

Stop the server, then copy the file:

```
cp instance/campus.db "backup-$(date +%Y-%m-%d).db"      # back up
cp backup-2026-11-10.db instance/campus.db               # restore
```

8.3 Reset

- Empty database, keep the venv: `python -m flask -app app init-db`, then `add-indexes` and `create-admin`.
- Everything from scratch: delete `venv/` and `instance/`, then run `./scripts/setup.sh`.
- Fresh demo data only: `./scripts/seed.sh`.

8.4 Inspect the data

Any SQLite browser can open the file, for example *DB Browser for SQLite* or the `sqlite3` command-line tool:

```
sqlite3 instance/campus.db
sqlite> .tables
sqlite> SELECT title, start_time FROM events ORDER BY start_time LIMIT 5;
sqlite> .quit
```

Do not edit the file while the server is handling requests.

9 Building the documentation

```
./scripts/build_docs.sh
```

This compiles six documents and places each PDF next to its source:

Source	Document
`report/report.tex`	Project report
`report/slides.tex`	Presentation slides (Beamer)
`docs/user_manual.tex`	User Manual
`docs/installation_guide.tex`	Installation & Operations Guide (this document)
`docs/developer_guide.tex`	Developer Guide
`docs/test_report.tex`	Test Plan & Report

Table 5: Documentation sources.

The documents in `docs/` share the style file `docs/campusdoc.sty` and use the screenshots in `report/figures/`. Without a local LaTeX installation, upload the `report/` and `docs/` folders together to Overleaf, keeping them side by side, and compile each `.tex` file with pdfLaTeX.

10 Troubleshooting

Problem	Cause and fix
`ERROR: Python 3 was not found.`	Install Python 3.10+ and make sure it is on the PATH. Open a new terminal afterwards.

Problem	Cause and fix
`$'\r': command not found` when running a script	The script has Windows (CRLF) line endings. The repository's `.gitattributes` prevents this on a fresh clone; to repair a copy, run `git rm --cached -r scripts && git checkout -- scripts` or convert with `dos2unix scripts/*.sh`.
`Permission denied` running `./scripts/x.sh`	Make the scripts executable: `chmod +x scripts/*.sh`, or run `bash scripts/x.sh`.
`flask.exe`: Permission denied (Windows)	A security policy blocks the launcher. The scripts already avoid it by using `python -m flask`; do the same when typing commands.
`ERROR: virtual environment not found.`	Run `./scripts/setup.sh`.
`ERROR: database not found.`	Run `./scripts/setup.sh` (it creates `instance/campus.db`).
`No admin account found` from `seed`	Create one with `create-admin`, or run `setup.sh`.
`Address already in use`	Another program uses port 5000 (on macOS often AirPlay Receiver). Use `PORT=8000 ./scripts/run.sh`.
`ModuleNotFoundError: No module named 'flask'`	The virtual environment is not active. Use the scripts, or `source venv/Scripts/activate` (Windows) / `source venv/bin/activate`.
`database is locked`	Another program (e.g. DB Browser) has the file open with unsaved changes. Close it or write the changes.
Logged out after restarting the server	Expected if `SECRET_KEY` changed between runs. Set a fixed value.
`No LaTeX installation found` from `build_docs.sh`	Install TeX Live or MiKTeX, or use Overleaf.

11 Deployment notes

The project is built for development and demonstration. Before exposing it to real users:

1. Set a long random `SECRET_KEY`, for example the output of `python -c "import secrets; print(secrets.token_hex(32))"`.
2. Do not use `--debug`; its interactive error pages must never be public.
3. Serve the app with a production WSGI server behind HTTPS, for example `waitress-serve --call app:create_app` on Windows, or `gunicorn "app:create_app()"` on Linux.
4. Change or remove the default admin password and do not load demo data.
5. Back up `instance/campus.db` regularly.

The project report lists further hardening as future work (CSRF tokens, atomic registration for the last seat).

12 Uninstalling

Everything lives inside the project folder. Stop the server and delete the folder. No system files, services or registry entries are created.